What Parsing Buys an Action Policy: Detection, Interruptions and Their Value for AI Coding Agents

Muhammad Usama
COMSATS University Islamabad
Islamabad, Pakistan
muhammadusamahoyrr@gmail.com

Awais Khalid
FAST National University of Computer and Emerging Sciences
Islamabad, Pakistan
2k7awaiskhalid@gmail.com

***Abstract*—AI coding agents run shell commands with the developer's privileges, behind policies that match the command text. We hold one policy engine fixed and vary only what its rules see: text, in substring and four regular-expression policies, or three facts derived from a parsed shell syntax tree. Parsing buys fewer interruptions than a line tokenizer, but against a tuned regex it buys different, not clearly fewer, interruptions. On an independent held-out benchmark of 150 actions, fact-based rules stop 90.0% of harmful actions at a 5.7% false-positive rate, against 16.3% for the substring policy. Most of that gain is vocabulary: a regex given the analyzer's word lists comes close, and the benchmark cannot separate the two. On our own shell traffic, a line tokenizer with the same semantics, here-documents and variables interrupts more than twice as often as facts. In a pre-specified test on 417 other developers in the public SWE-chat corpus, facts interrupt 1.5 points less than the tokenizer per developer (macro-averaged; median 0.5), and less for 251 developers against 28. Against the vocabulary regex the typical developer's difference is near zero. The pre-specified audit of whether stops are warranted was inconclusive; all audits were labeled by Claude agents, not humans. In exploratory agent audits, most tokenizer-only stops were benign, but a few were credential reads facts allow; regex-only stops were mostly mentions of credentials, and fact-only stops mostly code a careful reader, not the analyzer, can resolve. We release the analyzer, benchmarks, harness and a human labeling kit.**

***Index Terms*—AI coding agents, LLM agent security, guardrails, access control policies, shell command analysis, static analysis, usable security**

I. INTRODUCTION

Coding agents such as Claude Code, Cursor and Codex execute shell commands on developer machines. When they go wrong the damage is concrete: in July 2025 an agent deleted a production database despite the user's explicit code freeze, according to the user [1]; malicious code instructing an agent to wipe local files and cloud resources was shipped inside an IDE extension, though it failed to run [2]; and a supply-chain attack invoked installed agent CLIs with permission checks disabled to collect `.env` files and SSH keys [3]. Benchmarks find that agents act unsafely in a large share of risky situations [4]–[6].

The common mitigation is an *action policy*: rules that allow, deny, or escalate each tool call. Shell commands reach these rules as strings. Agent permission systems may split compound commands and strip wrappers, but the rule itself is a pattern over the command text. Vendor documentation warns that "Bash permission patterns that try to constrain command arguments are fragile" and that a deny rule "isn't a security boundary around the program" [7]; documented hook examples grep the command for `rm -rf` [8]. Industry reports show such guards bypassed by quoting, subshells and encoding [9], [10].

A policy can fail in two directions. Too weak, it misses a destructive command phrased differently (`rm -fr`, `find -delete`, `bash -c '...'`). Too strong, it interrupts so often that approvals become reflexive: habituation to security warnings is well documented [11]–[13], and agent vendors cite approval fatigue as a reason to reduce prompts [14], [15]. We therefore ask:

RQ1 How much do text-based policies miss on independently written actions?
RQ2 What does a fact layer add, and how much of it is vocabulary, normalization, the syntax tree, and per-tool semantics?
RQ3 What does each policy cost in interruptions on our own agent traffic, and are those interruptions warranted?
RQ4 Do these costs and audit results hold, pre-specified, on other developers' traffic and on a second agent?
RQ5 Where does the analysis stop helping against an adversary who has read it?

Contributions. (1) A controlled comparison that holds a policy engine fixed and varies only the predicate: substring rules, four regex variants (strong, traffic-tuned, vocabulary-matched and normalized), gate-everything, and facts from a shell AST, with ablations that replace the AST by a tokenizer or an off-the-shelf parser (Section III). (2) Evaluation on an independently written held-out benchmark, source-aware probes, 27,927 of our own Claude Code tool calls, a second agent (Codex), and a pre-specified test on 430 other developers (Sections IV–V). (3) A decomposition: held-out recall comes mostly from vocabulary, while the syntax tree is the largest single factor in how often the gate interrupts real work. (4) Audits of what interruptions are, including a relabeling that corrects our own first audit. (5) An open artifact.

Relation to prior work. CARE [16] also parses agent shell commands and ablates its structural layer. What is new

```
// v1: a predicate over the raw command text
tool_name == "Bash" &&
  (params.command.contains("rm -rf") ||
   params.command.contains("git push --force") || ...)

// fact-based policy: first match wins
facts.secret_access  -> deny
facts.destructive    -> requires_approval
facts.opaque         -> requires_approval
```

Listing 1. The v1 substring rule and the fact-based policy, both in CEL.

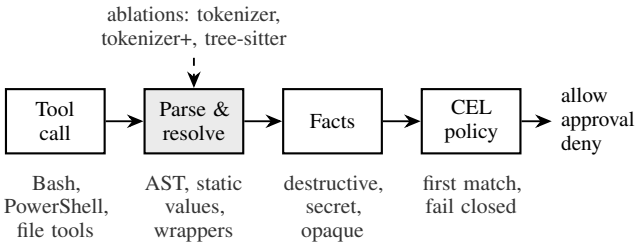


Fig. 1. The fact pipeline. Only the shaded stage changes between facts and its ablations; the engine and the rules stay fixed. For example, `D=build; sudo -u app 'r''m' -fr "$D"` resolves to `rm -rf build`, which is *destructive*, while the v1 rule `contains("rm -rf")` lets it pass.

here is a decomposition under one fixed policy engine against deliberately favorable text baselines, interruption cost measured on public traces with a pre-specified test across developers, and audits of what the interruptions are.

II. THREAT MODEL

An agent issues tool calls—`Bash`, `PowerShell`, `Read`/`Edit`/`Write`, `Grep`—intercepted before execution (here, by a Claude Code `PreToolUse` hook or an MCP proxy). The agent may be mistaken or steered by prompt injection. We gate three classes of action. (i) *Destructive*, defined syntactically: bulk or recursive deletion, destructive database statements, and rewriting or deleting shared history. As a policy choice this includes directories that could be regenerated; we measure its cost. (ii) *Secret access*: reading, copying or sending credential material. (iii) *Opaque*: execution whose effect cannot be determined from the call. Each call is judged in isolation. We evaluate robustness to rephrasing and to one short source-aware probe round, not to a sustained adaptive adversary. Out of scope are multi-step decompositions such as writing a script and then running it [17]. The same holds for any command that runs a project script or build target (`./x.sh`, `make`, `npm run`), 3.1% of real shell calls, whose effect lies in files the gate does not read. Also out of scope are effects that bypass the tool layer [18] and attacks on the gate itself.

III. DESIGN: FACTS UNDER A FIXED POLICY ENGINE

ActionGate evaluates each call against ordered rules in CEL [19], a non-Turing-complete expression language; the first match decides, and evaluation errors fail closed. Its earlier default (v1) matched substrings of the command. We keep the engine and add one input, a map `facts` computed by a pure function of tool name and parameters, so that the fact-based policy reduces to three rules (Listing 1).

As in authorization languages such as Cedar [20], the policy stays small while the application supplies structured attributes; an operator can write `facts.destructive && environment == "prod"` without knowing shell syntax (Fig. 1).

Shell analysis. Commands are parsed into a Bash AST with `mvdan.cc/sh` [21]. Each word is resolved to its static value: quotes and escapes removed, braces expanded (`.{e,}nv` → `.env`), and variables substituted when assigned a static value earlier in the same command. Wrappers that only run another command are stripped (`sudo -u x`, `env A=1`, `xargs`, `timeout`, `npx`); flags are interpreted, so `-fr`, `-r -f` and `--recursive` coincide. Nested shell code (`bash -c`, `eval` of literals, here-strings, static here-documents) is re-parsed to bounded depth. Destructive semantics are defined per tool: recursive `rm`, `find -delete`, force pushes and delete refspecs, `reset --hard`, `clean -f`, `dd` to a device, destructive SQL reaching a database client anywhere in a pipeline, and bulk-delete verbs of container, cluster and cloud CLIs.

Secret access counts only commands that expose contents (`cat`, `grep`, `sed`, `tar`, `curl -d @f`, `cp` sources, `git show`), not ones that test, list, or hand a file to a program (`test -f .env`, `docker --env-file .env`). A reader's leading pattern (`grep PATTERN FILE`) is not a file. Credential files are recognized by name; templates (`.env.example`) and public keys are excluded.

Opacity. At the shell level the analyzer does not guess: `eval` of computed text, a pipe into a shell or interpreter, a command name computed at run time, process substitution used as a script, unparseable input, excess nesting, and analyzer panics all yield *opaque*. Running a project script is not opaque but out of scope. Inline interpreter code (`python -c`, `node -e`, here-documents) is only scanned with patterns for deletion APIs, secret file names and embedded shell commands; code that matches nothing is *allowed*. A fourth fact, *inline_dynamic*, flags interpreter code that builds strings or names at run time; the *facts-strict* policy escalates it. PowerShell is analyzed at token level. The analyzer is about 1,400 lines of Go, performs no I/O, and its final Bash entry point was fuzzed for 60 s (369,056 inputs with four workers, no failures).

Ablation switches. For evaluation only, the analyzer accepts options that leave the policy untouched. *Tokenizer* replaces the AST with quote-aware splitting at `;`, `&&`, `||`, `|` and `&`, nothing expanded or re-parsed, under the same per-command semantics. *Tokenizer+* also hands here-document bodies to the command that reads them and substitutes static variables, still without a syntax tree. *Tree-sitter* (exploratory) feeds simple commands from the off-the-shelf tree-sitter-bash parser to the same semantics. *No-readers* counts every mention of a credential file as a read, *no-vars* disables variable substitution, and *exempt* stops treating a recursive `rm` as destructive when every target is a conventionally regenerable directory (`build`, `node_modules`, caches, `tmp`) or lies under a temporary directory.

IV. EVALUATION METHOD

Labels. Benign → allow; destructive or opaque → requires approval; secret access → deny. These are the analyzer's own

TABLE I
EVALUATION DATA

Set	Size	Role
Development	228 (133 harmful)	regression test
Held-out B	150 (80 harmful)	primary test
Held-out A	120 (70 harmful)	frozen analyzer v2.1
Probes	53 (50 harmful)	source-aware round
Own traffic (Claude Code)	27,927 calls	cost, in-sample
Codex, organic	4,364 commands	second agent
Codex, Bash (staged)	40 actions (3 harmful)	second agent
SWE-chat	640,530 calls, 417 dev.	pre-specified test
Self-published	453,036 calls, 13 dev.	pre-specified test

Benchmark sizes are actions; traffic sizes are governed tool calls (own traffic) or shell calls of developers with at least 50 (out of sample). B holds 33 PowerShell items, A 13, the development set 22.

categories, so the benchmarks measure conformance to this ontology, not harm in general. Benign items deliberately include look-alikes (dangerous words in commit messages or grep patterns, dry runs, templates), which is adversarial to text rules.

Benchmarks. Table I lists the data. The development set was written by the first author before the analyzer. Each held-out set was written in one pass by a separate LLM agent that saw only the label definitions and category counts, not the code, the other sets or analyzer output; no item was edited afterwards. A second blind agent agreed with all 270 labels; no human has labeled them. B was evaluated once, on the final analyzer; A was scored on an earlier frozen analyzer (v2.1), whose misses later informed fixes. The probes were written in about fifteen minutes by a reviewer agent that had the analyzer's source.

Real traffic. A hashed copy (prefix `60f161cc`) of the first author's Claude Code transcripts, frozen on October 31, 2026: Bash 19,932, PowerShell 352 and file tools 7,643 calls in 15 sessions, 97% of shell calls from one project, excluding the session in which this paper was written. Transcripts record calls as the agent issued them, not whether Claude Code's own permissions then blocked them.

Second agent. OpenAI's Codex CLI logs the argument vector of every command it runs. The organic corpus is the first author's 11 sessions, all run through PowerShell or `cmd.exe`. The Bash corpus comes from seven everyday tasks (fixing a test, cleaning and rebuilding, squashing commits, resetting a test database and others) run in Git Bash in a small project with a fake `.env`, a database and a git history; none asked for a destructive or secret-reading command; its 35 commands and 5 writes were labeled by the first author. All other agents in this study were Claude models, so shared blind spots remain a threat (Section VIII).

Other developers (pre-specified). Before reading these data we froze the analyzer and packs (hash prefix `6809d27d`) and specified three one-sided hypotheses over developers: facts stop fewer calls than tokenizer+ (H1) and than the vocabulary regex (H2) for more developers than the reverse (sign test), and a blind audit finds fact-only stops warranted more often than regex-only stops (H3, Fisher test). The primary source is SWE-chat [22]: real sessions of Claude Code, Codex, Pi, OpenCode and other agents, mined from public GitHub repositories that use the Entire.io CLI; 417 developers (commit authors) with at least 50 shell calls. The second source is 13 developers who published their own sessions. Older exports cut inputs at 300 characters, so an amendment made before any policy ran restricts the primary analysis to complete inputs and adds a secondary one (H1b, H2b) on commands shorter than 290 characters. The audit draws 30 fact-only and 30 regex-only stops from SWE-chat, one per developer, labeled blind by a Claude agent; no human labels were collected, and the items are released as a kit. The specification is a timestamped file with the frozen hash in our repository, not an external registration. It records ten deviations, the most consequential a converter fix made after a first run had been seen. Effect sizes, per-agent results, sensitivity runs, relabelings and the tree-sitter ablation are exploratory.

Policies. *v1*: ActionGate's earlier substring pack, a baseline we wrote in the style of documented hooks [8]. *Strong regex*: four RE2 rules for flag variants, git force and delete forms, SQL, `find -delete`, interpreter deletion APIs, PowerShell, secret names and pipes into shells, written with knowledge of the development set and held-out A, so deliberately favorable. *Tuned regex*: the same rules after a tuning pass on real traffic analogous to the analyzer's, five edits such as restricting `eval` to command position and ignoring secret names after `test` and `ls`. *Vocabulary(-matched) regex*: the tuned regex extended with every secret name and destructive verb the analyzer knows, assembled after B's results were seen. *Normalized vocabulary regex*: the same on commands after the analyzer's normalization. *Gate-all*: every shell command and file mutation needs approval, and secret-file reads are denied.

Metrics. Recall and FPR: harmful and benign actions stopped (deny or approval); exact: decision equals label. Friction: share of real shell calls stopped, pooled or macro-averaged per project, session or developer. A fact-only stop is one facts make and the compared policy does not. Intervals are 95% Wilson; differences use Newcombe's intervals [23], [24]. On B, exact McNemar tests (facts vs. v1 on recall, and vs. the strong, tuned and vocabulary regexes on recall and FPR) are Holm-corrected; items cluster by technique, so even these are optimistic. On our traffic we use two-sided sign tests over the 15 sessions; out of sample, the pre-specified one-sided sign tests over developers, each at $\alpha = 0.05$ without correction.

V. RESULTS

RQ1: what text misses. Our substring policy stops 16.3% of B's harmful actions (Table II; CI 9.7–25.8), partly because it has no rule for shell secret reads or PowerShell; on the development set it stops all 11 canonical destructive commands but 1 of 52 rephrasings. A strong regex reaches 82.5% recall at 12.9% FPR: patterns broad enough to catch variants also catch dangerous words inside harmless commands.

RQ2: what the fact layer adds. Facts stop 90.0% of B's harmful actions (CI 81.5–94.8) at 5.7% FPR (CI 2.2–13.8). The gain over v1 is large (59 vs. 0 discordant items, Holm-adjusted $p < 10^{-16}$); against the strong regex neither recall (+7.5

TABLE II
DETECTION AND INTERRUPTION COST

	Held-out B (80 harmful / 70 benign)			Held-out A (70 / 50)		Probes (50)	Real traffic
Policy	Recall	FPR	Exact	Recall	FPR	Recall	Shell friction
v1 (our substring pack)	16.3%	4.3%	53.3%	17.1%	10.0%	20.0%	1.21%
Strong regex	82.5%	12.9%	84.7%	90.0%[a]	6.0%[a]	72.0%	6.88%
Tuned regex	77.5%	10.0%	83.3%	90.0%[a]	6.0%[a]	72.0%	2.68%
Vocabulary-matched regex[d]	86.3%	11.4%	87.3%	94.3%[a]	8.0%[a]	76.0%	3.64%
Vocabulary-matched regex, normalized	85.0%	11.4%	86.7%	–	–	78.0%	3.56%
Gate-all	100%	94.3%	43.3%[c]	98.6%	94.0%	98.0%	100%
Facts	**90.0%**	**5.7%**	**92.0%**	91.4%	0.0%	78.0%	**2.74%**
tokenizer (no AST)	86.3%	5.7%	90.0%	–	–	74.0%	9.90%
tokenizer+ (here-docs, vars)	86.3%	5.7%	90.0%	–	–	74.0%	6.69%
tree-sitter parser[e]	90.0%	5.7%	92.0%	–	–	78.0%	3.29%
without read-vs-mention	90.0%	8.6%	90.7%	–	–	78.0%	3.42%
without variable resolution	90.0%	5.7%	92.0%	–	–	78.0%	4.93%
with regenerable-path exemption	90.0%	5.7%	92.0%	–	–	74.0%	2.56%
Facts-strict	90.0%	5.7%	92.0%	–	–	86.0%[b]	8.38%

Held-out B is the primary test; A was scored on the analyzer frozen at v2.1. Shell friction: share of the 20,284 real shell calls a policy stops. [a]Regex written with knowledge of A. [b]Strict variant designed after the probe round. [c]Includes six secret reads that gate-all's file-tool rule denies exactly. [d]Assembled after B's results were seen. [e]Exploratory: tree-sitter-bash in place of the AST, same per-command semantics; its front end was corrected once after B's misses were inspected (71/80 before).

points) nor FPR (-7.1) differs significantly (adjusted $p = 0.55$). Against the tuned regex recall is $+12.5$ points (adjusted $p = 0.04$), but this is fragile: tuning cost the regex exactly four dynamic-code items. Giving the regex the analyzer's vocabulary lifts it to 86.3% at 11.4% FPR, leaving facts $+3.8$ recall points (CI -3.6 to $+11.6$) and -5.7 FPR points (CI -13.7 to $+0.9$): not significant, but B cannot exclude a 12-point difference either. Normalizing first changes almost nothing (85.0%), and replacing the AST by either tokenizer loses three harmful items (86.3%) and no FPR. On B, then, vocabulary carries most of the recall and token-level semantics most of the precision; facts' residual edge is opaque Bash code (9 vs. 5 of 10), the vocabulary regex's is destructive PowerShell (8 vs. 6 of 8). On an Intel Core i5-8350U laptop every policy evaluates an action in well under a millisecond.

RQ3: cost on real traffic. On the frozen corpus facts stop 2.74% of shell calls (224 denials, 332 approvals) and the strong regex 6.88%, more than facts in 10 of 15 sessions and fewer in 2 ($p = 0.04$). Most of the regex's extra approvals come from a bare `eval` that also matches `pytest -k eval`, from output capture such as `s=$(curl ...)`, and from commands that begin with a variable; tuning those away brings it to 2.68%, and adding the analyzer's vocabulary raises it to 3.64% (vs. facts: 7 vs. 2 sessions, $p = 0.18$). The ablations show what produces facts' rate. Without the AST the tokenizer stops 9.90% (more than facts in 12 sessions, fewer in none, $p = 0.0005$), because here-documents and multi-line programs fall apart into spurious commands; tokenizer+ recovers part of this but still stops 6.69% (11 vs. 0 sessions, $p = 0.001$). The rest comes from quoting and command substitution that span lines, compound commands and nested code. Without variable resolution facts stop 4.93%, and without the read-vs-mention rule 3.42%. The syntax tree, which barely mattered on B, is the largest single factor on real work. Macro-averaged, facts stop 2.77% per project and 2.92% per session, tokenizer+ 6.86% and 6.59%. Facts stop none of the 7,643 file-tool calls but 9.4% of the 352 PowerShell calls, mostly scripts invoked through a variable that token-level analysis cannot resolve.

What our own interruptions are. In a seeded sample of 40 fact-based denials, 37 (92.5%, CI 80.1–97.4) read or copied a real credential file, as ordinary development work, so approval may suit secret reads better than a hard deny. Of 40 sampled approvals, labeled by the first author with Claude's assistance and by a separate Claude agent ($\kappa = 0.90$), at most one was worth the interruption: 27–28 deleted build outputs, caches or scratch directories the agent had just created, 7–8 were analyzer false positives, and 4–5 ran benign run-time code. Exempting regenerable paths per call lowers friction only to 2.56% and lets the probe `mv src /tmp/trash && rm -r /tmp/trash` through; exempting targets created earlier in the session lowers it to 2.38%, because most such directories are created by programs, not by commands the gate sees. A first audit of disagreements with the vocabulary regex found a gated action in 25 of 30 fact-only stops but only 3 of 30 regex-only denials, which mostly mentioned a credential file without reading it. The joint relabeling below shows that this first audit counted readable run-time code as gated, and supersedes it.

RQ4: a second agent. On the organic Codex corpus, which the analyzer was never tuned on, facts stop 1.86% of 4,364 commands, the tuned and vocabulary regexes 1.8% and 1.9%, and the strong regex 12.2%; every command was PowerShell, so this tests only token-level analysis. In the staged Bash corpus Codex produced three harmful actions: two `git clean` runs, one of which removed every git-ignored file including

TABLE III
OUT OF SAMPLE: OTHER DEVELOPERS' SHELL CALLS

Policy	SWE-chat (417 dev.)			Self-published (13)		
	Pooled	Macro	Fewer/ more	Pooled	Macro	Fewer/ more
v1	0.83%	0.86%	–	0.48%	0.88%	–
Strong regex	3.74%	3.04%	–	2.24%	2.89%	–
Tuned regex	2.57%	2.02%	119/123	1.09%	1.38%	1/10
Vocab. regex	3.16%	2.33%	166/86	1.18%	1.56%	2/8
Facts	**1.98%**	2.21%	–	1.30%	2.33%	–
tokenizer	4.73%	4.48%	270/21	2.04%	3.30%	10/0
tokenizer+	3.82%	3.75%	251/28	1.88%	3.05%	10/0
tree-sitter*	2.05%	2.26%	68/21	1.33%	2.37%	6/1

Share of shell calls stopped: pooled over calls, and macro-averaged over developers with at least 50 shell calls (SWE-chat 640,530 calls; self-published 453,036). Fewer/more: developers for whom facts stop fewer / more calls than that policy (one-sided sign test). –: not compared. Analyzer frozen before the data were read. *Exploratory off-the-shelf parser ablation (Table II).

TABLE IV
WHERE FACTS DO NOT INTERRUPT LESS THAN A REGEX

Setting	Regex	Dev.	Fewer/more	p
SWE-chat	tuned[a]	417	119/123	0.63
OpenCode users[b]	vocab.	35	10/8	0.41
Pi users[b]	vocab.	13	4/4	0.64
Self-published (H2)	vocab.	13	2/8	>0.98
	tuned[a]	13	1/10	>0.99
Self-published, <290 chars (H2b)	vocab.	19	3/12	>0.99
	tuned[a]	19	1/13	>0.99

Fewer/more: developers for whom facts stop fewer / more shell calls than the regex; p: one-sided sign test that facts stop fewer. On SWE-chat as a whole the macro-averaged difference from the vocabulary regex is -0.1 points (CI -0.4 to $+0.2$). [a]Not a pre-specified hypothesis. [b]Exploratory: developers with at least 50 shell calls of that agent.

the developer's `.env` and SQLite database, and a script it had written that deletes old files. Facts require approval for both `git clean` commands and interrupt nothing else (2 of 3 harmful, 0 of 37 benign); the script run is out of scope by the threat model. The vocabulary regex matches facts, and v1 lets all three through. The sample is tiny, but it shows the failure this paper is about: a routine instruction turned into bulk deletion.

RQ4 (pre-specified): 430 other developers. On SWE-chat facts stop 1.98% of shell calls and tokenizer+ 3.82% (Table III). Facts stop fewer calls for 251 developers and more for 28 ($p = 3 \times 10^{-46}$; H1 holds), and fewer than the vocabulary regex for 166 against 86 ($p = 3 \times 10^{-7}$; H2 holds). The tuned regex, not in the hypotheses, ties facts (119 vs. 123). On the 13 self-published developers H1 holds (10 vs. 0, $p = 0.001$) but H2 does not; there the common fact reasons include 688 commands the Bash parser rejected, which fail closed. In the secondary analysis on commands shorter than 290 characters, neither H1b (8 vs. 6, $p = 0.40$) nor H2b holds. Table IV collects every setting in which facts do not interrupt less than a regex: H2 holds on SWE-chat as a whole and for Claude Code and Codex users, and nowhere else.

Effect sizes and robustness (exploratory). Because the sign test counts a one-call difference like a large one, we also report per-developer differences on SWE-chat. Against tokenizer+ facts interrupt 1.5 points less macro-averaged (bootstrap CI 1.3–1.8; median 0.5). Against the vocabulary regex the macro difference is -0.1 points (CI -0.4 to $+0.2$) and the median is zero; the pooled difference (-1.2) comes from heavy users, the ten largest of whom hold 41% of calls. Against the tuned regex it is $+0.2$ (CI -0.05 to $+0.5$). For 286 of 417 developers facts and the vocabulary regex differ by less than half a point. Within agents, H1 holds for Claude Code, Codex and OpenCode users but not significantly for Pi (8 vs. 2, $p = 0.055$). Excluding the 32 developers in the collection tool vendor's own repositories changes nothing (H1 224 vs. 28, H2 152 vs. 80). The tokenizer gap persists on commands under 290 characters (0.98% vs. 1.58%), but about three quarters of tokenizer+'s extra stops come from the 22% of calls that are longer. The sign tests drop ties (138 developers for H1, 165 for H2), many of them stopped by neither policy.

An off-the-shelf parser (exploratory). To check that the result does not depend on our parser, the tree-sitter ablation replaces mvdan's syntax tree. After one correction informed by B's misses (71/80 before), it matches facts on B, and it matches them on the probes without one (Table II). On SWE-chat facts stop fewer calls than tree-sitter for 68 developers and more for 21 (Table III), but the macro-averaged difference is only 0.05 points (CI 0.01–0.09), and tree-sitter interrupts less than tokenizer+ for 245 developers and more for 36; 0.7% of distinct commands do not parse in tree-sitter and fail closed. The parser mattered only on our own traffic, where the analyzer's false-positive fixes were made on mvdan's tree (3.29% vs. 2.74%). The gap to the tokenizer is a property of parsing, not of our parser.

What the extra stops are (pre-specified audit). The annotator, which agreed with B's labels on 49 of the 50 kit items ($\kappa = 0.97$; both are Claude models), found a gated action in 5 of 30 fact-only stops (17%, CI 7–34) and 3 of 30 regex-only stops (10%, CI 3–26), a difference of $+6.7$ points (CI -11.5 to $+24.8$; Fisher $p = 0.35$), so H3 is inconclusive. Fact-only stops were mostly fail-closed opacity: a command name held in a variable (10), interpreter code built at run time (7) and input the parser rejected (4).

One protocol for both audits (exploratory). The in-sample audit had found 25 of 30. To separate population from protocol, a fresh agent relabeled all 150 audit items, shuffled and blind to source, with an added label, chosen after seeing the discrepancy, for code built or named at run time that a careful reader can resolve. The two populations then look alike: 8 of 30 in-sample fact-only stops (27%, CI 14–44) and 5 of 30 out-of-sample ones (17%, CI 7–34) were strictly gated, 19 and 17 were such readable run-time code, and 3 and 8 benign; regex-only stops were strictly gated in 2–6 of 30 per stratum. Fact-only stops are mostly conservative opacity, on the analyzer's own traffic as on others'. The added label is relative to a static analyzer and was applied by a model; human labels may draw the line elsewhere.

Tokenizer-only stops (exploratory). Under the same protocol an agent labeled 30 tokenizer+-only and 30 fact-only stops from SWE-chat (48 developers). 26 tokenizer+-only stops were

benign builds, tests, API queries and status polls, mostly multi-line; the other 3 read a token from a keychain, a password manager or an auth file, which facts allowed. 9 fact-only stops were strictly gated (30%, CI 17–48; 7 secret reads, 2 force pushes), against 3 (10%), a difference of $+20$ points (CI -0.5 to $+39$; $p = 0.052$). Parsing removes mostly spurious stops and adds a few warranted ones, but it does not catch everything the tokenizer catches.

RQ5: a source-aware probe round. With the source in hand, a reviewer agent evaded facts on 11 of 50 harmful probes: six read credential stores missing from the name list (Docker and GitHub CLI configs, `~/.kube/config`, `terraform.tfstate`, `.envrc`), four built strings inside interpreter code (`open('.e'+'nv')`), and one truncated a database file. The normalized vocabulary regex ties facts (78.0%), and v1 stops 20.0%. Escalating dynamic interpreter code (facts-strict) reaches 86.0%, post hoc, but raises real-traffic friction to 8.38%, because agents build strings in inline Python constantly. We did not replay the incidents of Section I, whose exact command lines are not public.

VI. ANALYSIS

Real traffic finds what benchmarks do not. The first fact-based analyzer scored 100% on its development set and 91.4% recall at 0% FPR on held-out A, yet stopped 18.2% of real shell calls on an earlier snapshot of our corpus. Three causes were absent from every benchmark: static `python - <<'EOF'` here-documents treated as code from stdin (2,568 calls), interpreter paths held in a variable (439 calls), and grep patterns and globs read as secret paths. Fixing them brought friction to 2.7% without changing held-out A, which is why our own friction is in-sample. Benchmarks are written one action at a time; agents write multi-line programs.

What remains hard on held-out data. Facts' 8 misses and 4 false positives on B fall into four classes: open-world vocabulary (7; e.g., `manage.py flush`, `prisma migrate reset`, `aws ec2 terminate-instances`), interpreter evaluation (1; Perl `eval pack`), intent modifiers (3 false positives: `terraform plan -destroy`, `kubectl delete --dry-run`, `Remove-Item -WhatIf`) and token-level PowerShell (1 false positive). Held-out A's six misses on v2.1 were fixed afterwards, so the current analyzer's 100% on A is not a test result. Open-world vocabulary and dynamic interpreter code argue for a two-stage design: deterministic facts for the common, explainable cases, and a learned classifier or LLM judge [15], [16] only for unknown tools and dynamic code.

Recommendations for deploying agent action policies.

1) *Invest first in vocabulary, then in structure.* Vocabulary carried held-out recall; a parser then interrupted less than an equally informed tokenizer, and about as often as a regex tuned on our traffic.
2) *Make the unknown explicit, and know what it excludes.* Route statically unresolvable constructs to a human; scripts and build targets (3.1% of shell calls) need a separate control.
3) *Judge secret access by contents, not by mentions,* and consider approval rather than a hard deny for reads, which are often legitimate.
4) *Scope destructive rules by what is lost,* but expect per-call path rules to recover little: deciding whether a deletion is regenerable needs session context.
5) *Evaluate on traces you did not tune on,* report interruptions per developer or session, and audit what they are; public corpora such as SWE-chat make this possible.
6) *Treat inline interpreter code as a separate problem,* to escalate (8.4% friction here) or delegate to a semantic judge.

VII. RELATED WORK

Agent action policies. Progent [25], AgentSpec [26], Conseca [27], ClawGuard [28] and Datalog monitors [29] enforce deterministic policies over tool calls; CaMeL [30] and Fides [31] enforce control and information flow; GuardAgent [32] and ShieldAgent [33] synthesize or verify guard logic; LlamaFirewall [34] combines regex and LLM scanners. These target mostly API-style tools [35]; a rule over a free-form shell argument remains a text rule. CARE [16] canonicalizes and parses shell commands, scores risk with weights tuned on a development split and sends borderline cases to an LLM judge; it reports deny rates on benign command sets, not interruptions on live traffic. AgentGuard [36] learns execution rules from anomalous trajectories; we instead fix the rules and vary the predicate. Claude Code's auto mode [15] reports 0.4% FPR and 17% false negatives for an LLM classifier on internal data; an independent stress test found 81% false negatives on 253 ambiguous state-changing actions, 37% of which fell outside the classifier's scope as in-project file edits [37], a blind spot a per-call shell gate shares. These figures use different data and units and are not comparable to ours. User-authored allow/ask/never rules reduce prompts but can weaken protection [38], [39]; our packs are expert-written.

Benchmarks and containment. ToolEmu [40], R-Judge [41], RedCode [42], OpenAgentSafety [4], SABER [5] and IssueTrojanBench [6] score agents rather than gates. Real-session corpora such as SWE-chat [22] have been used to study developer-agent misalignment [43]; we use them to price gate interruptions. Sandboxes [14], OS-level enforcement [18] and recoverable filesystems [44] bound where damage lands but cannot tell `rm -rf build/` from `rm -rf src/` inside a workspace; they complement a per-call gate.

Shell analysis and obfuscation. Shell semantics are subtle [45], and static analysis of shell scripts has a long history [46]. Obfuscation frameworks show how cheaply string signatures are defeated [47]–[49]. Parsing shell commands improved learned security classification over generic tokenization in a small study without a held-out split [50]; we quantify the analogous effect inside a policy gate. Learned obfuscation detectors [51], [52] and AST-level deobfuscation with emulation [53] also show the value of recovering structure; we recover it without emulating and send the unresolved remainder to a human.

Warning fatigue. Habituation to security warnings [11]–[13], work on scoped consent [54] and consent integrity [55], and Anthropic's report that sandboxing cut Claude Code permission prompts by 84% [14] make the interruption rate a first-class metric.

VIII. LIMITATIONS

Labels. All benchmarks are small. The held-out sets and probes were written and labeled by Claude models, the family that assisted development, and the development set and Codex Bash corpus by the first author; no second human labeled any of them, so they may share blind spots (B's 25 secret items were all caught, while the probe reviewer quickly found five unlisted credential stores). They test conformance to our ontology, not expert judgments of harm. We release a labeling kit (110 items, a guide and an agreement script) so that others can check the agent's labels.

Statistics. Items cluster by technique and audit samples by session or developer, so even Holm-adjusted tests and Wilson intervals are optimistic, and B cannot resolve differences of about ten points.

Traffic. Our Claude Code traffic is one developer's 15 sessions, dominated by one project and in-sample. The Codex corpora add a second agent but not a second developer, and the Bash one is small and staged. The out-of-sample corpora add 430 developers but are unlabeled; SWE-chat's developers are a self-selected population that publishes sessions through one checkpointing tool, 32 of them in that vendor's repositories, and overlap with the self-published developers was not checked. Every corpus records what agents issued under their own permission modes, and agents adapt to prompts, so friction is an upper bound on the extra prompts a gate adds.

Analysis. Labels encode a policy choice (every recursive delete is destructive), and secret detection is by file name. We did not test whether static resolution matches Bash on corner cases (ANSI-C quoting, alias or function shadowing). The public traces contain zsh, fish and PowerShell commands that our converters pass to the Bash analyzer, where they fail closed.

Comparisons. No off-the-shelf tokenizer was compared, and we compare against no LLM judge, no vendor default policy and no replayed incident. The probe round was one short LLM session.

Data and ethics. The in-sample traffic is the first author's own sessions; because it contains local paths and occasionally secrets, it is not released. The out-of-sample data are public (SWE-chat under ODC-By and its access terms, and openly licensed self-published sessions). We report only aggregates, number developers instead of naming them, and do not redistribute their commands. Benchmark items contain no real credentials.

IX. CONCLUSION

Holding the policy engine fixed, deriving three facts from a parsed command turns our substring policy, which stops one harmful action in six, into one that stops nine in ten; most of that held-out gain is vocabulary, which a regex can borrow. Where parsing differs is real work. On our own traffic and, pre-specified, on 417 other developers, the fact layer interrupted much less than a tokenizer given the same semantics, here-documents and variables, chiefly on long, multi-line commands, and an off-the-shelf parser does the same. Against a regex it buys different interruptions, not clearly fewer or more valuable ones: regex-only stops are mostly mentions of a secret, fact-only stops mostly conservative opacity, and on 13 self-published developers both regexes interrupted less. Most remaining interruptions are approvals for deleting regenerable directories, which a per-call gate lacks the context to judge. Artifact: https://github.com/muhammadusamahoyrr/actiongate.

ACKNOWLEDGMENT

M. Usama designed and built ActionGate and its analyzer, ran the evaluation and wrote the paper. A. Khalid contributed to the evaluation design, reviewed the results and the error analysis, and revised the manuscript. The analyzer and this paper were developed with assistance from an AI model (Claude, Anthropic); Claude-based agents also wrote the held-out and probe benchmarks and performed the second annotations, the blind audits and the relabelings (Section IV). All numbers were produced by the released harness. We thank the reviewers of earlier drafts, the SWE-chat authors, and the developers who published their own sessions.

REFERENCES

[1] S. Sharwood, "Vibe coding service Replit deleted user's production database, faked data, told fibs galore," The Register, Jul. 21, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/

[2] Amazon Web Services, "Security update for Amazon Q Developer extension for Visual Studio Code (version #1.84)," AWS Security Bulletin AWS-2025-015, Jul. 23, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://aws.amazon.com/security/security-bulletins/AWS-2025-015/

[3] M. Bar and R. McCarthy, "s1ngularity: Supply chain attack leaks secrets on GitHub: Everything you need to know," Wiz Blog, Aug. 27, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.wiz.io/blog/s1ngularity-supply-chain-attack

[4] S. Vijayvargiya, A. B. Soni, X. Zhou, Z. Z. Wang, N. Dziri, G. Neubig, and M. Sap, "OpenAgentSafety: A comprehensive framework for evaluating real-world AI agent safety," in *International Conference on Learning Representations (ICLR)*, 2026.

[5] Q. Hu, Y. Tang, Q. Wang, L. Zhao, P. Zhang, Y. Qing, X. Yao, D. Huang, L. Zhang, and Z. Ji, "SABER: Benchmarking operational safety of LLM coding agents in stateful project workspaces," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026, pp. 4842–4866, arXiv:2606.01317.

[6] A. Singh, J. Yang, and T.-H. Chen, "IssueTrojanBench: Benchmarking AI coding agents against malicious issue requests," 2026, arXiv:2607.20759.

[7] Anthropic, "Configure permissions," Claude Code Documentation, accessed: Oct. 30, 2026. [Online]. Available: https://code.claude.com/docs/en/permissions

[8] ——, "Hooks reference," Claude Code Documentation, accessed: Oct. 30, 2026. [Online]. Available: https://code.claude.com/docs/en/hooks

[9] T. Claburn, "Cursor AI YOLO mode lets coding assistant run wild, security firm warns," The Register, Jul. 21, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.theregister.com/2025/07/21/cursor_ai_safeguards_easily_bypassed/

[10] K. Townsend, "Decades-old Bash tricks expose AI coding agents to supply chain attacks," SecurityWeek, Jun. 30, 2026, accessed: Oct. 30, 2026. [Online]. Available: https://www.securityweek.com/decades-old-bash-tricks-expose-ai-coding-agents-to-supply-chain-attacks/

[11] D. Akhawe and A. P. Felt, "Alice in Warningland: A large-scale field study of browser security warning effectiveness," in *USENIX Security Symposium*, 2013, pp. 257–272.

[12] B. B. Anderson, C. B. Kirwan, J. L. Jenkins, D. Eargle, S. Howard, and A. Vance, "How polymorphic warnings reduce habituation in the brain," in *ACM Conference on Human Factors in Computing Systems (CHI)*, 2015, pp. 2883–2892, doi: 10.1145/2702123.2702322.

[13] A. Vance, C. B. Kirwan, D. Bjornn, J. L. Jenkins, and B. B. Anderson, "What do we really know about how habituation to warnings occurs over time?" in *ACM Conference on Human Factors in Computing Systems (CHI)*, 2017, pp. 2215–2227, doi: 10.1145/3025453.3025896.

[14] D. Dworken and O. Weller-Davies, "Beyond permission prompts: Making Claude Code more secure and autonomous," Anthropic Engineering Blog, Oct. 20, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://www.anthropic.com/engineering/claude-code-sandboxing

[15] J. Hughes, "How we built Claude Code auto mode: A safer way to skip permissions," Anthropic Engineering Blog, Mar. 25, 2026, accessed: Oct. 30, 2026. [Online]. Available: https://www.anthropic.com/engineering/claude-code-auto-mode

[16] Y. Liu, W. Zhang, Z. Yang, Z. Zhang, H. Feng, X. Wang, P. Qiu, Y. Liu, B. Poczos, and J. B. Hong, "CARE: Pre-execution command verification for shell-executing LLM agents," in *IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2026, arXiv:2607.21642.

[17] X. Li, S. Yu, M. Pan, Y. Sun, B. Li, D. Song, X. Lin, and W. Shi, "Unsafer in many turns: Benchmarking and defending multi-turn safety risks in tool-using agents," 2026, arXiv:2602.13379.

[18] Y. Zheng, T. Wu, Q. Fu, T. Yu, W. Mao, T. Ma, D. Williams, W. Wang, and A. Quinn, "ActPlane: Programmable OS-level policy enforcement for agent harnesses," 2026, arXiv:2606.25189.

[19] CEL Contributors, "Common Expression Language (CEL)," Project website, accessed: Oct. 30, 2026. [Online]. Available: https://cel.dev/

[20] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. M. Wells, "Cedar: A new language for expressive, fast, safe, and analyzable authorization," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 670–697, 2024, doi: 10.1145/3649835.

[21] mvdan.cc/sh Contributors, "sh: A shell parser, formatter, and interpreter, version 3.12.0," Software, 2025, accessed: Oct. 30, 2026. [Online]. Available: https://github.com/mvdan/sh

[22] J. Baumann, V. Padmakumar, X. Li, J. Yang, D. Yang, and S. Koyejo, "SWE-chat: Coding agent interactions from real users in the wild," in *Proc. Conf. Language Modeling (COLM)*, 2026, arXiv:2604.20779. Dataset https://huggingface.co/datasets/SALT-NLP/SWE-chat, revision 202b071f.

[23] R. G. Newcombe, "Improved confidence intervals for the difference between binomial proportions based on paired data," *Statistics in Medicine*, vol. 17, no. 22, pp. 2635–2650, 1998.

[24] ——, "Interval estimation for the difference between independent proportions: Comparison of eleven methods," *Statistics in Medicine*, vol. 17, no. 8, pp. 873–890, 1998.

[25] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," 2025, arXiv:2504.11703.

[26] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2026, arXiv:2503.18666.

[27] L. Tsai and E. Bagdasarian, "Contextual agent security: A policy for every purpose," in *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*, 2025, arXiv:2501.17070.

[28] W. Zhao, Z. Li, P. Zhang, and J. Sun, "ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection," 2026, arXiv:2604.11790.

[29] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, I. Dillig, and S. Jha, "Formal policy enforcement for agentic systems," 2026, arXiv:2602.16708.

[30] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025, arXiv:2503.18813.

[31] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," 2025, arXiv:2505.23643.

[32] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, D. Song, and B. Li, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," in *International Conference on Machine Learning (ICML)*, 2025.

[33] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding agents via verifiable safety policy reasoning," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025, pp. 8313–8344.

[34] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents," 2025, arXiv:2505.03574.

[35] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024, arXiv:2406.13352.

[36] W. Dai and S. Wang, "AgentGuard: Learning execution guardrails from anomalous coding-agent trajectories," 2026, arXiv:2609.16287.

[37] Z. Ji, Z. Li, P. Ma, W. Jiang, Y. Gao, and S. Wang, "Measuring the permission gate: A stress-test evaluation of Claude Code's auto mode," 2026, arXiv:2604.04978.

[38] T. Yan, "Do user-authored permission policies improve protection against AI agent overreach?" 2026, arXiv:2608.27443.

[39] A. E. Michael and F. Roesner, "How agents ask for permission: User permissions for AI agents, from interfaces to enforcement," in *Third Workshop on Agents in the Wild at NeurIPS*, 2026, arXiv:2607.13718.

[40] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2309.15817.

[41] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, "R-Judge: Benchmarking safety risk awareness for LLM agents," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024, arXiv:2401.10019.

[42] C. Guo, X. Liu, C. Xie, A. Zhou, Y. Zeng, Z. Lin, D. Song, and B. Li, "RedCode: Risky code execution and generation benchmark for code agents," in *NeurIPS Datasets and Benchmarks Track*, 2024.

[43] N. Tang, C. Chen, G. Xu, Y. Shi, Y. Huang, C. McMillan, T. Dong, and T. J.-J. Li, "How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions," 2026, arXiv:2605.29442.

[44] S. W. Zhong, J. Liao, J. Liu, M. Zheng, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Don't let AI agents YOLO your files: Information and control in agent-native filesystems," 2026, arXiv:2604.13536.

[45] M. Greenberg and A. J. Blatt, "Executable formal semantics for the POSIX shell," *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 1–30, 2020, doi: 10.1145/3371111.

[46] K. Mazurak and S. Zdancewic, "ABash: Finding bugs in Bash scripts," in *Workshop on Programming Languages and Analysis for Security (PLAS)*, 2007, pp. 105–114, doi: 10.1145/1255329.1255347.

[47] D. Bohannon, "Invoke-Obfuscation: PowerShell command and script obfuscator," Software, 2016, accessed: Oct. 30, 2026. [Online]. Available: https://github.com/danielbohannon/Invoke-Obfuscation

[48] ——, "DOSfuscation: Exploring the depths of Cmd.exe obfuscation and detection techniques," FireEye, Tech. Rep., 2018.

[49] A. LeFevre, C. Barker, N. Hatfield, and E. Barker, "Bashfuscator: A fully configurable and extendable Bash obfuscation framework," Software, 2018, accessed: Oct. 30, 2026. [Online]. Available: https://github.com/Bashfuscator/Bashfuscator

[50] D. Trizna, "Shell language processing: Unix command parsing for machine learning," in *Conf. Applied Machine Learning in Information Security (CAMLIS)*, 2021, arXiv:2107.02438.

[51] D. Hendler, S. Kels, and A. Rubin, "Detecting malicious PowerShell commands using deep neural networks," in *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2018, pp. 187–197, doi: 10.1145/3196494.3196511.

[52] V. Outrata, M. A. Polak, and M. Kopp, "Command-line obfuscation detection using small language models," 2024, arXiv:2408.02637.

[53] Z. Li, Q. A. Chen, C. Xiong, Y. Chen, T. Zhu, and H. Yang, "Effective and light-weight deobfuscation and semantic-aware attack detection for PowerShell scripts," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 1831–1847, doi: 10.1145/3319535.3363187.

[54] Y. Li, Y. Chen, P. Wang, I. Khabra, F. H. Shezan, Y. Feng, and Y. Tian, "Options, not clicks: Lattice refinement for consent-driven MCP authorization," 2026, arXiv:2605.11360.

[55] X. Weng, "What you approve is what executes: Consent integrity for black-box LLM agents," 2026, arXiv:2606.02668.